Lecture 8

- Email application
- DNS

E-mail

Three major components:

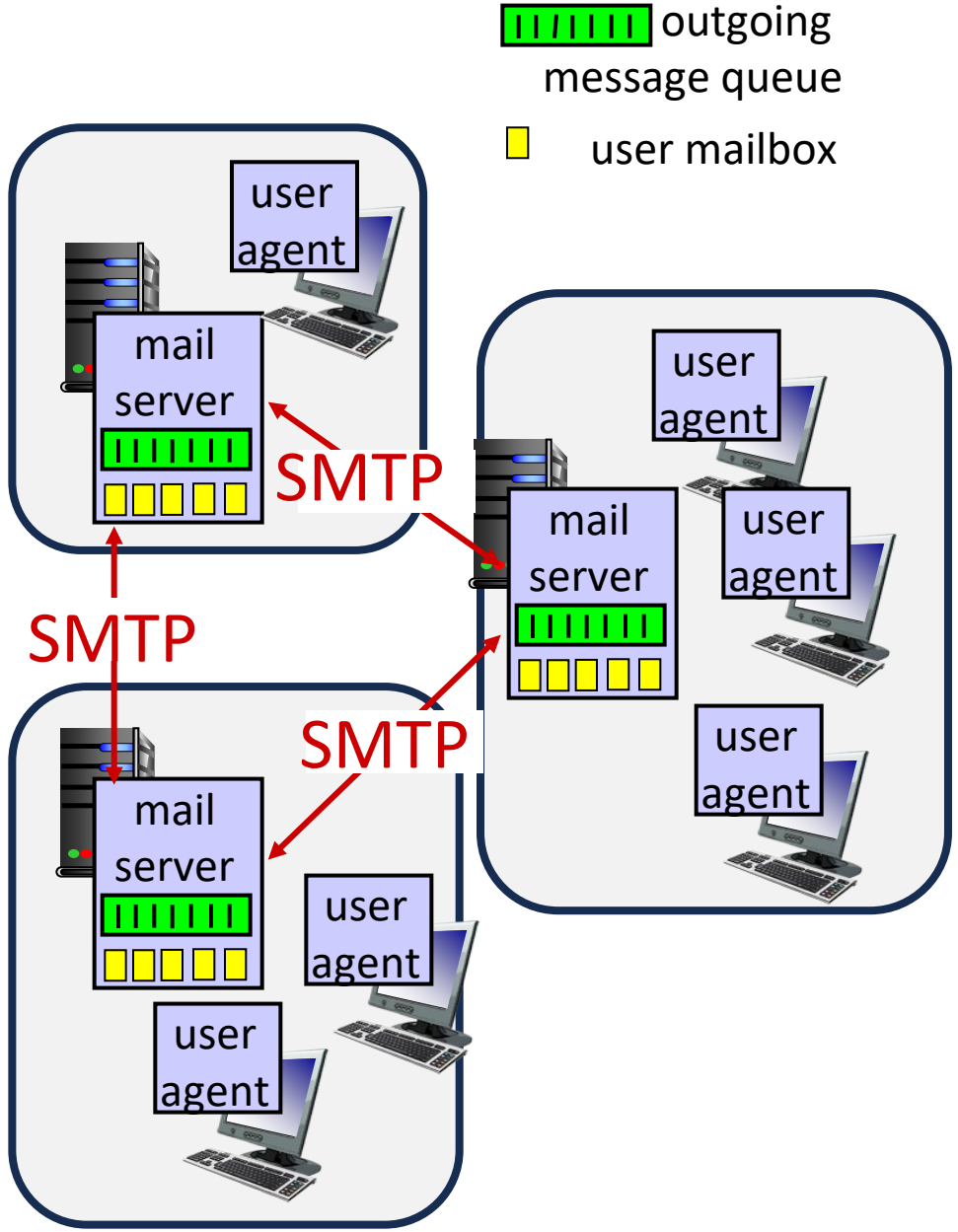
(1) user agents

(2) mail servers

(3) Simple mail transfer protocol: **(SMTP)**

(1) User Agent

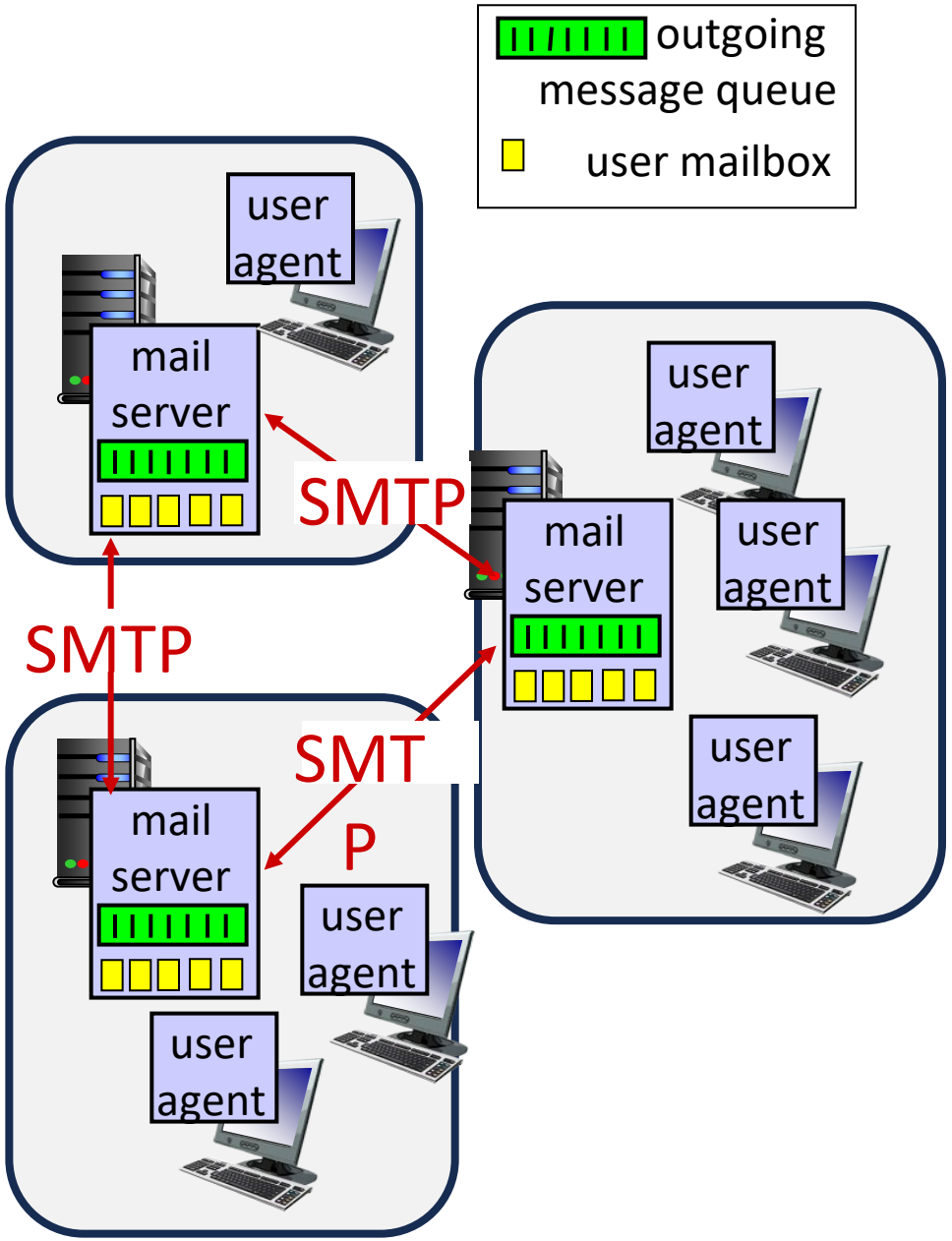
- a.k.a. “mail reader”
- composing, editing, reading mail messages
- e.g., Outlook, iPhone mail client



(2) mail servers

mail servers:

- *mailbox* contains incoming messages for user
- *message queue* of outgoing (to be sent) mail messages



(3) SMTP

- *SMTP protocol* between mail servers to send email messages
  - client: sending mail server
  - "server": receiving mail server
- Uses TCP between client (mail server initiating connection) to server, port 25
- Direct transfer: sending server (acting like client) to receiving server
- Three phases of transfer
  - handshaking (greeting)
  - transfer of messages
  - closure
- Command/response interaction (like HTTP)
  - commands: ASCII text
  - response: status code and phrase

Difference with HTTP:

HTTP: pull

SMTP: push

Scenario: Alice sends e-mail to Bob

1) Alice uses UA to compose e-mail message “to” bob@someschool.edu

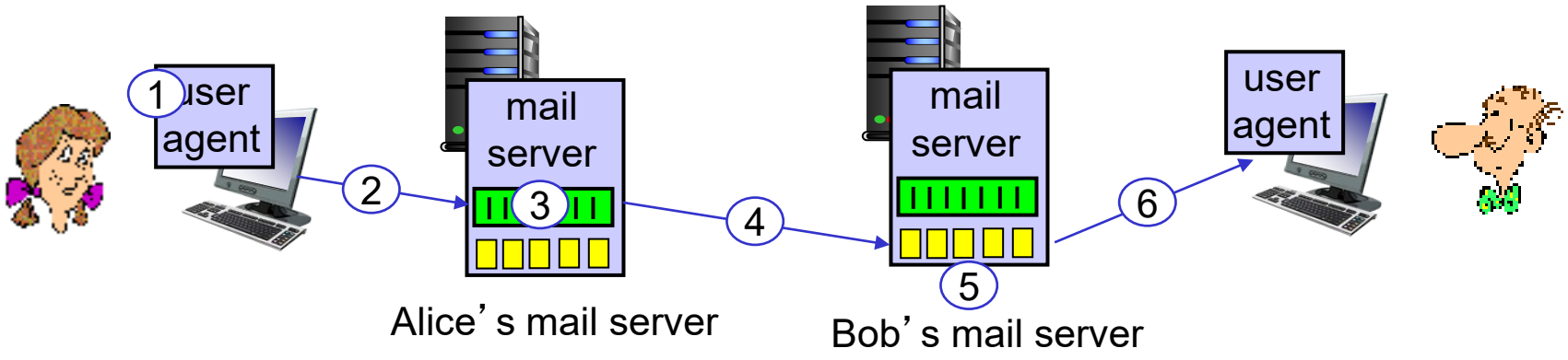
2) Alice’s UA sends message to her mail server; message placed in message queue

3) client side of SMTP opens TCP connection with Bob’s mail server

4) SMTP client sends Alice’s message over the TCP connection

5) Bob’s mail server places the message in Bob’s mailbox

6) Bob invokes his user agent to read message



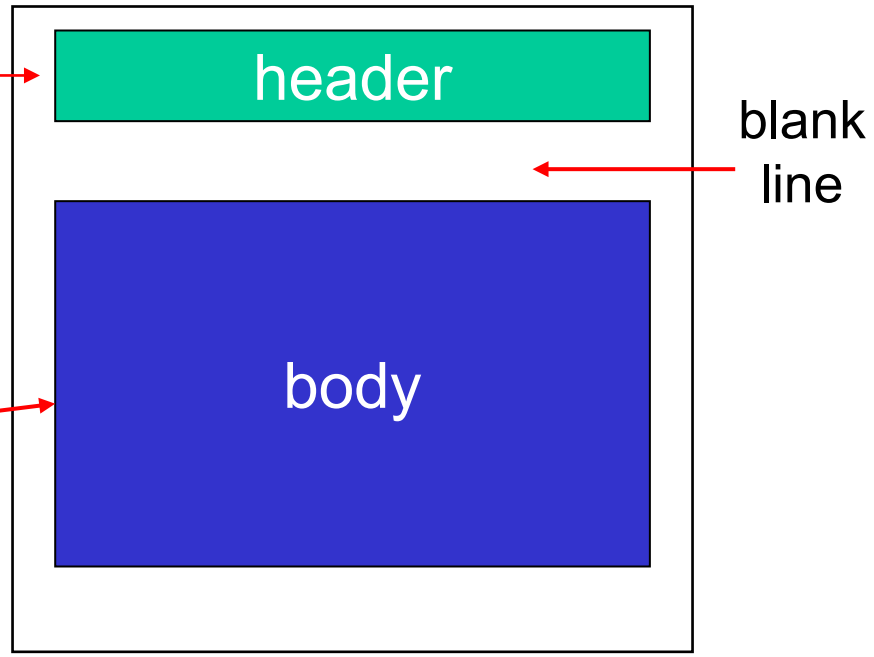
E-Mail message format

SMTP: protocol for exchanging e-mail messages, defined in RFC 821 (like HTTP)

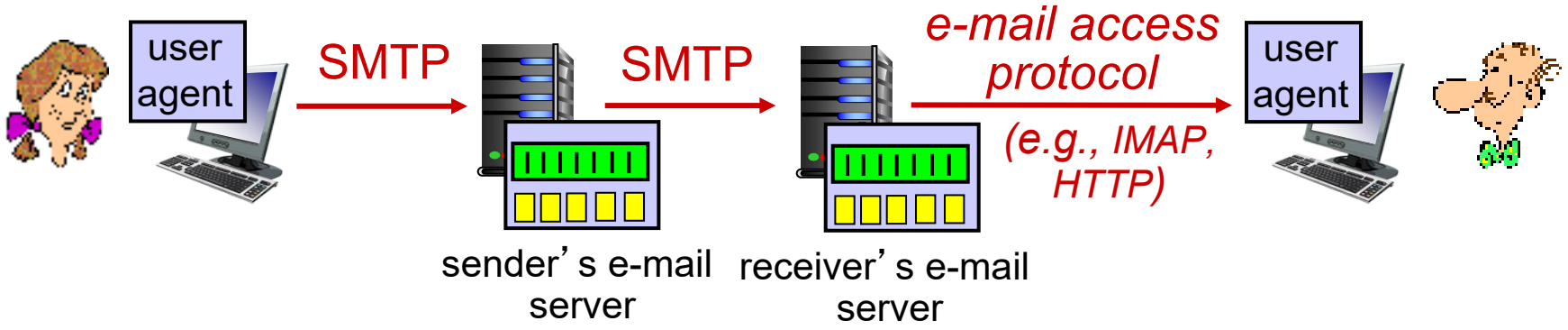
BUT, we need to define *syntax* for e-mail message itself (like HTML) (RFC 822)

- header lines, e.g.,
  - To:
  - From:
  - Subject:

  these lines, within the body of the email message area different from SMTP MAIL FROM:, RCPT TO: commands!
- Body: the "message" , ASCII characters only



Mail access protocols



- SMTP: **delivery/storage of e-mail messages to receiver's server**
- **mail access protocol: retrieval from server**
  - POP: Post Office Protocol [RFC 1939]
    - Authorization (agent - server) and download
  - IMAP: Internet Mail Access Protocol [RFC 3501]:
    - More features (more complex)
    - Manipulation of stored msgs on server (retrieval, deletion, folder on the server)
  - HTTP: gmail, Hotmail, etc. provides web-based interface on top of SMTP (to send), IMAP (or POP) to retrieve e-mail messages

POP3 and IMAP

POP3

- Traditional POP ➔ "download and delete" mode
- POP3 "download-and-keep": copies of messages on different clients
- POP3 uses port 110 (unencrypted), while secure POP3 uses port 995

IMAP

- keeps all messages in one place: at server
- allows user to organize messages in folders
- keeps user state across sessions:
  - names of folders and mappings between message IDs and folder name
- IMAP uses port 143 (unencrypted) and port 993 (encrypted)

DNS: Domain Name System

DNS: Domain Name System

people: many identifiers:

- SSN, name, passport #

Internet hosts, routers:

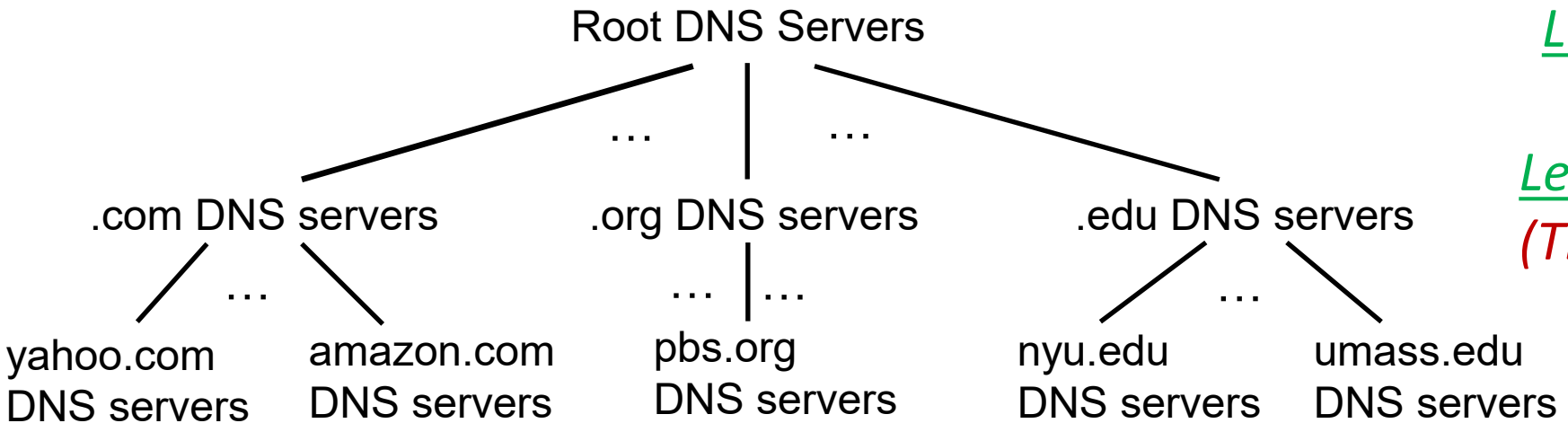
- IP address (32 bit) - used for addressing datagrams
- "name", e.g., eecs.ku.edu - used by humans

Q: how to map between IP address and name, and vice versa ?

A: Domain Name System: distributed database implemented in hierarchy of many *name servers*

DNS: Core Internet function, implemented as application-layer protocol to *resolve* names (address/name translation)

DNS: a distributed, hierarchical database



Level 1: Root

Level 2: Top Level Domain (TLD)

Level 3: Authoritative

Client wants IP address for www.amazon.com; 1st approximation:

- client queries root server to find .com DNS server
- client queries .com DNS server to get amazon.com DNS server
- client queries amazon.com DNS server to get IP address for www.amazon.com

Level 1: root name servers

- official, contact-of-last-resort by name servers that can not resolve name
- *incredibly important* Internet function
  - Internet couldn't function without it
- ICANN (Internet Corporation for Assigned Names and Numbers) manages root DNS domain

13 logical root name "servers" worldwide each "server" replicated many times (~200 servers in US)



Level 1: root name servers

List of Root Servers

HOSTNAME	IP ADDRESSES	OPERATOR
a.root-servers.net	198.41.0.4, 2001:503:ba3e::2:30	Verisign, Inc.
b.root-servers.net	170.247.170.2, 2801:1b8:10::b	University of Southern California, Information Sciences Institute
c.root-servers.net	192.33.4.12, 2001:500:2::c	Cogent Communications
d.root-servers.net	199.7.91.13, 2001:500:2d::d	University of Maryland
e.root-servers.net	192.203.230.10, 2001:500:a8::e	NASA (Ames Research Center)
f.root-servers.net	192.5.5.241, 2001:500:2f::f	Internet Systems Consortium, Inc.
g.root-servers.net	192.112.36.4, 2001:500:12::d0d	US Department of Defense (NIC)
h.root-servers.net	198.97.190.53, 2001:500:1::53	US Army (Research Lab)
i.root-servers.net	192.36.148.17, 2001:7fe::53	Netnod
j.root-servers.net	192.58.128.30, 2001:503:c27::2:30	Verisign, Inc.
k.root-servers.net	193.0.14.129, 2001:7fd::1	RIPE NCC
l.root-servers.net	199.7.83.42, 2001:500:9f::42	ICANN
m.root-servers.net	202.12.27.33, 2001:dc3::35	WIDE Project

Location of servers: https://root-servers.org/

From: https://www.iana.org/domains/root/servers

nslookup command

```
C:\Users\mhashemi>nslookup -type=ns .
Server:  stephens.ittc.ku.edu
Address:  129.237.125.220

Non-authoritative answer:
(root)  nameserver = a.root-servers.net
(root)  nameserver = g.root-servers.net
(root)  nameserver = j.root-servers.net
(root)  nameserver = e.root-servers.net
(root)  nameserver = h.root-servers.net
(root)  nameserver = k.root-servers.net
(root)  nameserver = f.root-servers.net
(root)  nameserver = l.root-servers.net
(root)  nameserver = m.root-servers.net
(root)  nameserver = c.root-servers.net
(root)  nameserver = i.root-servers.net
(root)  nameserver = b.root-servers.net
(root)  nameserver = d.root-servers.net

i.root-servers.net      internet address = 192.36.148.17
k.root-servers.net      internet address = 193.0.14.129
d.root-servers.net      internet address = 199.7.91.13
e.root-servers.net      internet address = 192.203.230.10
b.root-servers.net      internet address = 170.247.170.2
c.root-servers.net      internet address = 192.33.4.12
g.root-servers.net      internet address = 192.112.36.4
f.root-servers.net      internet address = 192.5.5.241
j.root-servers.net      internet address = 192.58.128.30
h.root-servers.net      internet address = 198.97.190.53
a.root-servers.net      internet address = 198.41.0.4
m.root-servers.net      internet address = 202.12.27.33
l.root-servers.net      internet address = 199.7.83.42
i.root-servers.net      AAAA IPv6 address = 2001:7fe::53
k.root-servers.net      AAAA IPv6 address = 2001:7fd::1
```

IP addresses for all root servers

TLD and authoritative servers

Level 2: Top-Level Domain (TLD) servers:

- responsible for .com, .org, .net, .edu, and all top-level country domains, e.g.: .cn, .uk, .fr, .ca, .jp
- VeriSign Company: authoritative registry for .com, .net TLD
- EduCause: .edu TLD

Level 3: Authoritative DNS servers:

- organization's own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts
- can be maintained by organization or service provider

Local DNS name servers

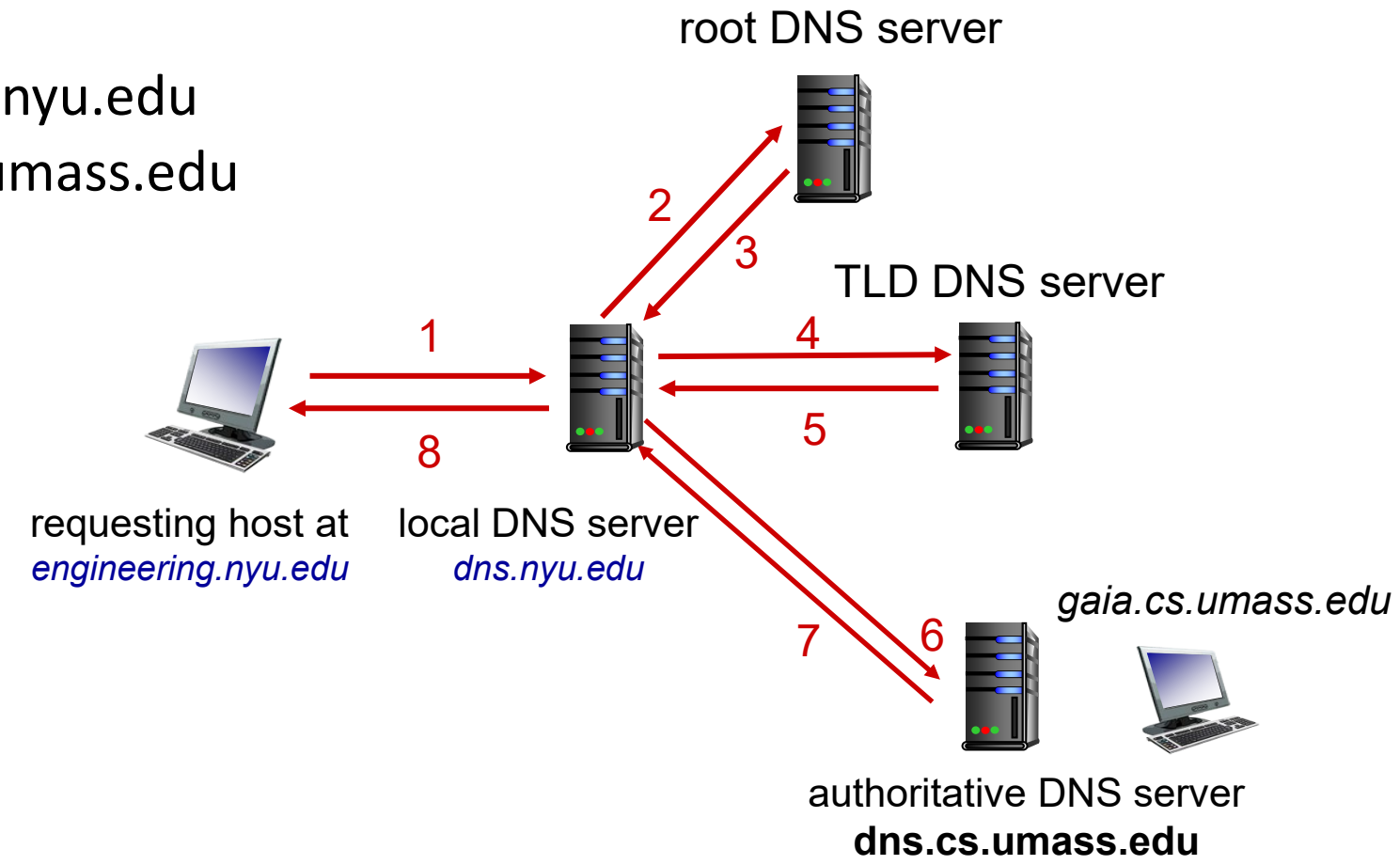
- Does not strictly belong to hierarchy
- Each ISP (residential ISP, company, university) has one
  - also called “default name server”
- When host makes DNS query, query is sent to its local DNS server
  - has local cache of recent name-to-address translation pairs (but may be out of date!)
  - acts as proxy, forwards query into hierarchy if needed

DNS name resolution: *iterated query*

Example: host at engineering.nyu.edu wants IP address for gaia.cs.umass.edu

Iterated query:

- contacted server replies with name of server to contact
- "I don't know this name, but ask this server"

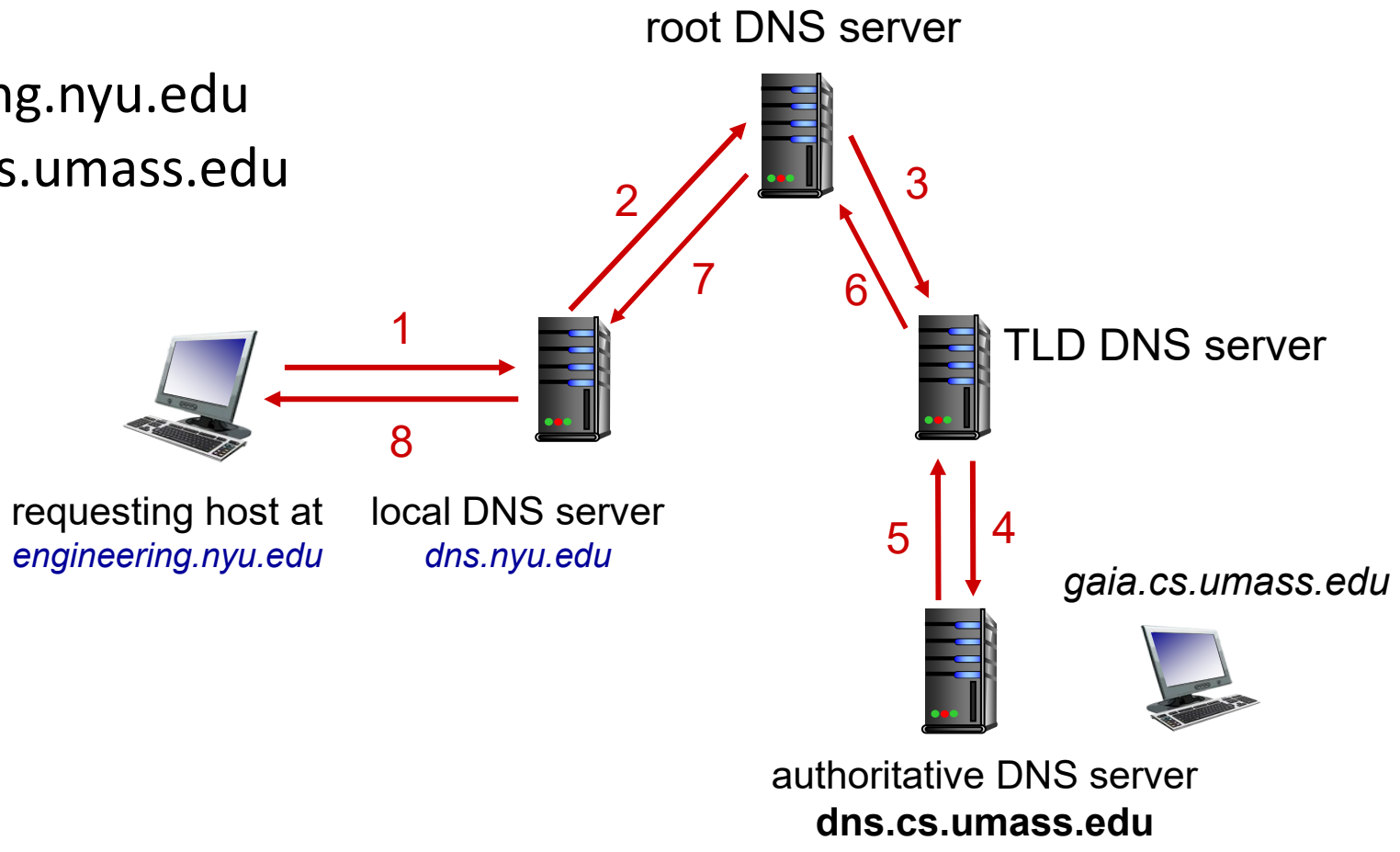


DNS name resolution: *recursive query*

Example: host at engineering.nyu.edu wants IP address for gaia.cs.umass.edu

Recursive query:

- puts burden of name resolution on contacted name server
- heavy load at upper levels of hierarchy



DNS: Domain Name System

Q: Why not centralize DNS?

- single point of failure
- traffic volume
- distant centralized database
- maintenance

A: doesn‘t scale!

- Comcast DNS servers alone: 600B DNS queries per day

Caching, Updating DNS Records

- once (any) name server learns mapping, it *caches* mapping
  - cache entries timeout (disappear) after some time (TTL)
  - TLD servers typically cached in local name servers
    - thus root name servers not often visited
- cached entries may be *out-of-date* (best-effort name-to-address translation!)
  - if name host changes IP address, may not be known Internet-wide until all TTLs expire!

DNS: services

DNS services

- *Most popular/intuitive service:* hostname to IP address translation
- *Other services*
  - host aliasing: canonical vs. alias names
  - mail server aliasing
  - load distribution ➔ replicated Web servers: many IP addresses could correspond to one name
  - For example, depending on the geolocation, www.google.com could be mapped to different IP address.

DNS records

DNS: distributed database storing **resource records (RR)**

RR format: (`name, value, type, ttl`)

type=A

- `name` is hostname
- `value` is IP address

type=NS

- `name` is domain (e.g., ku.edu)
- `value` is the authoritative name server for this domain

type=CNAME

- `name` is alias name for some "canonical" (the real) name
- www.ibm.com is really servereast.backup2.ibm.com
- `value` is canonical name

type=MX

- `value` is name of mail-server associated with `name`

Real-world example

- Akamai *EdgeKey* Servers




```
┌──(kali㉿kali)-[~]
└─$ nslookup -type=cname www.hulu.com
Server:         192.168.189.2
Address:        192.168.189.2#53

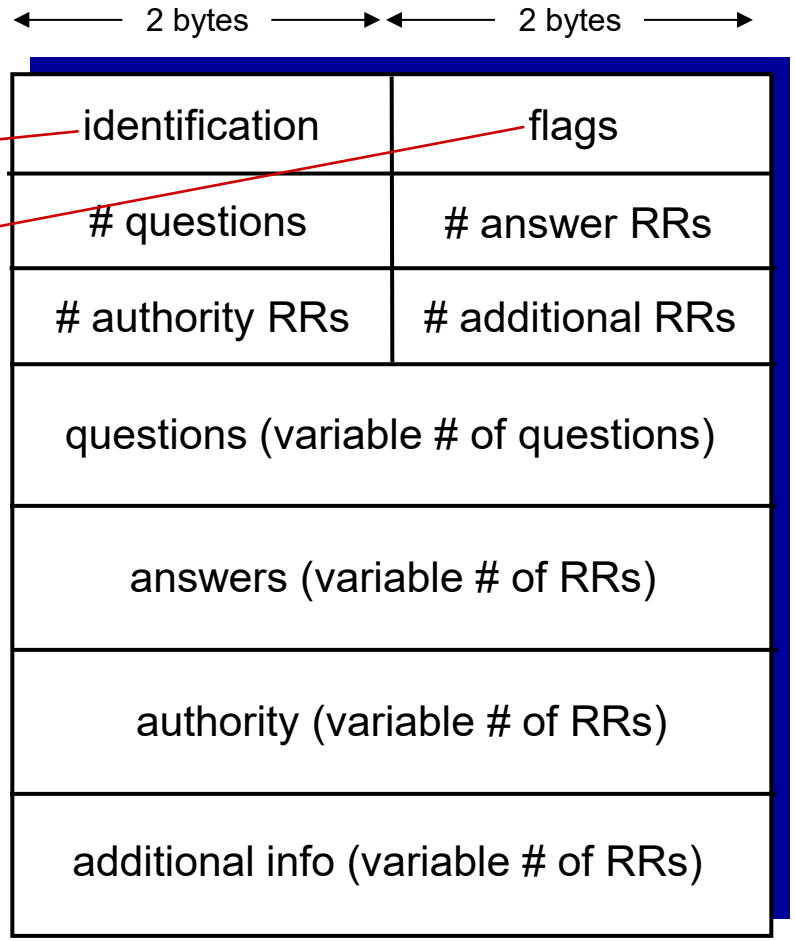
Non-authoritative answer:
www.hulu.com    canonical name = wildcard.hulu.com.edgekey.net.
```

DNS protocol messages

DNS *query* and *reply* messages, both have same *format:*

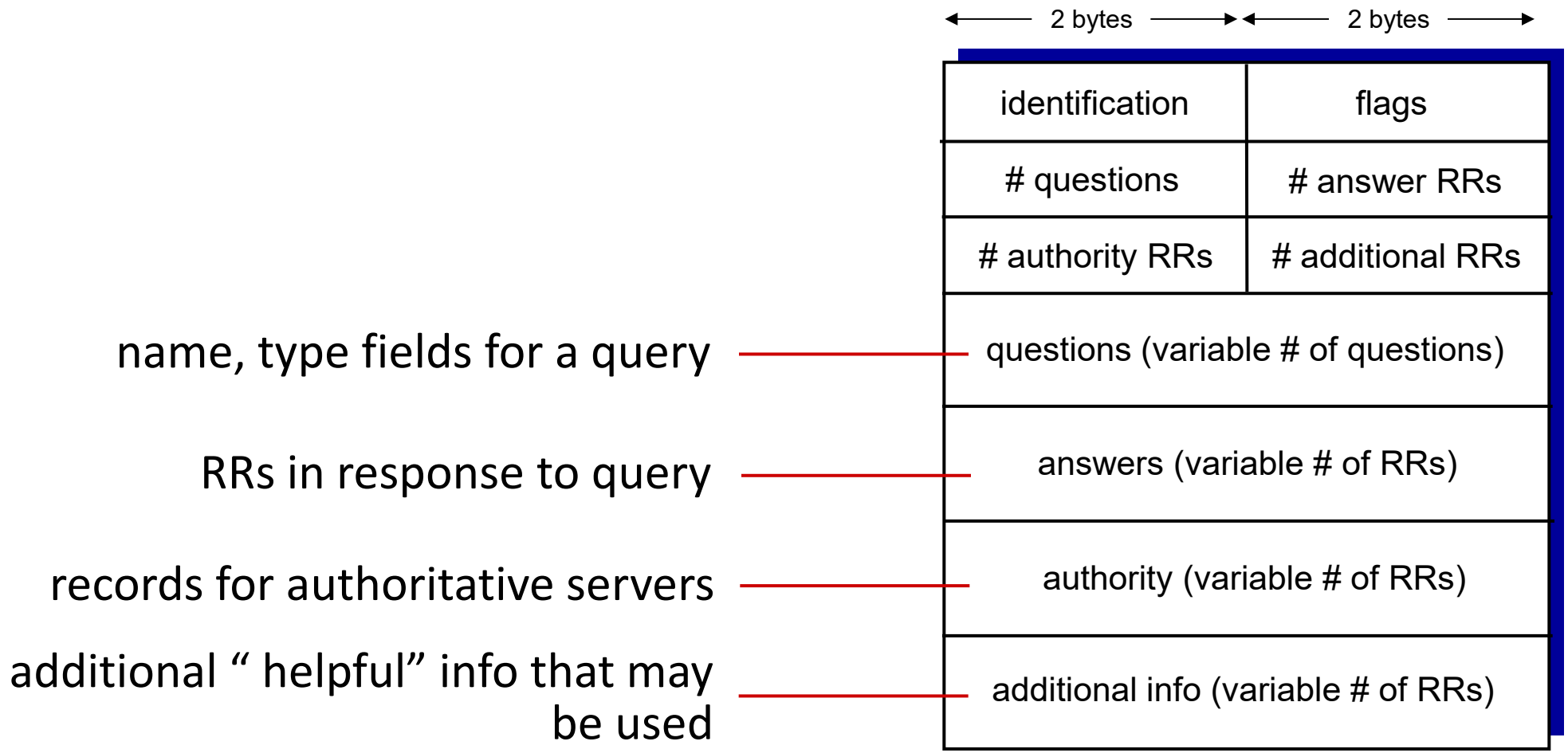
message header:

- identification: 16 bit # for query, reply to query uses same #
- flags:
  - query or reply
  - recursion desired
  - recursion available
  - reply is authoritative



DNS protocol messages

DNS *query* and *reply* messages, both have same *format:*



Inserting records into DNS

Example: new startup “Network Utopia”

- register name networkuptopia.com at *DNS registrar* (e.g., VeriSign)
  - provide names, IP addresses of authoritative name server (primary and secondary)
  - registrar inserts NS, A RRs into .com TLD server:

    `(networkutopia.com, dns1.networkutopia.com, NS)`

    `(dns1.networkutopia.com, 212.212.212.1, A)`
- create local DNS server locally with IP address `212.212.212.1`
  - Insert type A record for www.networkuptopia.com ➔ for example, 212.212.71.4
  - Insert type MX record for networkuptopia.com email server

DNS security

DDoS attacks

- bombard root servers with traffic
  - not successful to date
  - local DNS servers cache IPs of TLD servers, allowing root server bypass
- bombard TLD servers
  - potentially more dangerous

Redirect attacks

- man-in-middle
  - Intercept DNS queries
- DNS poisoning
  - send bogus replies to DNS server, which caches

Exploit DNS for DDoS

- send queries with spoofed source address: target IP

DNSSEC
[RFC 4033]